

AstroJSON: A New Data Interchange Format for Maps of Space

1. Terminology

“**GeoJSON**” or “**RFC7946**” refers to [IETF Request for Comments 7946](#) and the ubiquitous geospatial data interchange format detailed therein.

“**Body**” refers to any physical object in space, including planets, stars or comets.

“**Existing standards**” refer to a collection of existing formats which could be used in part to store, transport, or represent the locations of bodies in space. This term includes, but is not limited to, GeoJSON, cartesian coordinate systems, and methods for calculating the positions of distant stars.

Conventions and definitions for JSON used as described in [RFC7159](#), and for GeoJSON in [RFC7946](#).

The key words "MUST", "MUST NOT", "REQUIRED", "SHALL", "SHALL NOT", "SHOULD", "SHOULD NOT", "RECOMMENDED", "MAY", and "OPTIONAL" in this section are to be interpreted as described in [RFC 2119](#).

2. Abstract

The rapidly growing space sector lacks a standardized Data Interchange Format for mapping, resulting in fragmented, hard-to-use data that requires specialized knowledge. As funding increases for space programs in the UK, EU and the US, we will increasingly need to produce maps of space. Maps are essential data visualisation tools; they are intuitive, and help detail, clarify and draw relationships between objects in space. Space maps help research, conduct and analyse space programs, while also visually communicating their value to the public and media. Despite their importance, we lack the infrastructure to map space in a standardised manner. Namely, the lack of a Data Interchange Format means that space data is stored in disparate formats which require custom software and domain knowledge, and is therefore difficult to use.

This paper seeks to address this issue, describing the need for a data interchange format for space mapping, and proposing a new “AstroJSON” format which exemplifies required properties, including flexible coordinate origins, expanded dimensionality, time as a dimensional component, high-precision data types, and astronomical database compatibility.

3. Introduction

Interchange formats are essential enablers of mapping efforts. Interchange formats provide a standardised structure for representing data, in a format that is independent of the instrument that was used to capture it, as well as any down-stream software or programming language which consumes it. Without such a standard, it is very difficult to integrate disparate datasets, share, communicate or consume them. For terrestrial mapping, GeoJSON ([RFC7946](#)) is the dominant interchange format; it encodes data as text in a universal JSON ([RFC7159](#)) schema, and is adopted by mapping software (e.g., [D3](#)) and is supported as an egress data type by geographic [databases](#).

There is no interchange format for space maps. As a result, any effort to map the properties or relationships between bodies in space requires manual process or conversion of disparate and expert data sets, in a process which is laborious, error-prone, and inaccessible to non-domain-experts. This issue is highly relevant since we are seeing historic increases in space funding in the [UK](#), [EU](#) and the [US](#), and we will need standardised space maps to plan, analyse, and visualise missions across the solar system, as well as deep-space exploration, and to communicate the value of these missions to the public and media. Therefore, it is important that we develop a data interchange format to enable creating maps of space.

This paper aims to document the requirements for an interchange format for space mapping, and propose a new “AstroJSON” standard to meet them. The scope of this paper concerns the *format* in which space map data is stored, transported or processed, and not the *mechanics* for how space map data might be produced (e.g., physics instruments, measurements, analysis, or simulation), nor the *display* of map data on a screen (e.g., projection/3D-2D transformation). This paper will also focus on viewpoint-independent dynamically rendered “vector” maps, and not pre-rendered viewpoint-dependent “raster” maps that use composite imagery and are not suitable for space mapping (e.g., projections of the night sky, Google Earth, see **Table 1**).

GeoJSON

GeoJSON is the existing standard for geographic mapping, and as such is a good basis for elucidating a space equivalent. This sub-section will outline the GeoJSON standard.

GeoJSON is a JSON-based format for encoding geographic data for storage, transport and use. GeoJSON represents geographical features, such as roads and borders, as one of seven supported “geometries” (see **Table 3** for all geometries). Points denote a single geodetic, latitude-longitude, coordinate position on Earth’s surface (World Geodetic System 1984, [WGS84](#)). Polygons string together coordinates to draw a shape. For example, the geographic centre of London can be mapped as a point at 51.50° N 0.13° W, and its full extent can be mapped as a polygon with a series of coordinates drawing a boundary around the city. These geometries are similar to geographic data formats such as [shapefiles/GIS](#), but the GeoJSON encoding specifies that features are represented in a JSON ([RFC7159](#)) schema, with several named properties which must be provided including “type” and “geometry”, and precise valid values including “Feature” and “Polygon”.

Figure 1. Minimal example of a GeoJSON feature with a point geometry.

```
json
{
  "type": "Feature",
  "properties": {
    "name": "City of London"
  },
  "geometry": {
    "type": "Polygon",
    "coordinates": [[
      [-0.0935, 51.5134],
      [-0.0763, 51.5194],
      [-0.0724, 51.5112],
      ...
      [-0.0935, 51.5134] // closes the ring
    ]]
  },
}
```

The unique value of GeoJSON is in its portability and universal nature. JSON is text-based and encoding data this way ensures it can be used across any programming language. Furthermore, its geometry system is flexible, allowing it to be used for many purposes including mapping, data visualisation, and analysis. Consequently, GeoJSON has become the de-facto interchange format for storing, transporting, analysing and visualising geographic maps on the internet, and is an IETF standard (RFC 7946). For example, the UK's [Office for National Statistics](#) supports the GeoJSON format so that disparate data sets can be compared and combined without manual processing, such as overlaying population data over a map of the UK's borders.

In summary, GeoJSON is the ubiquitous format for building maps of the Earth. Geographic features are represented as geometries on Earth's surface using ellipsoidal latitude-longitude coordinates. The next section will contrast this system against the requirements for space mapping.

Space maps

This section will discuss how a GeoJSON-like standard could be adopted for space maps, and detail the modifications that would need to be made. GeoJSON is adopted as the basis point for this new standard since it is an existing standard, is familiar, and its feature and geometry systems are tried-and-tested. The systems outlined in the GeoJSON standard are also technically extendable and modifiable for our purposes, though clauses in the standards documentation clearly state such extension must be labelled as an entirely new standard (RFC7946 sections 7, 8). We will therefore adopt a policy of "least-required modification", and discuss the requirements for a new Data Interchange Format for mapping space.

Coordinate system

Coordinate reference system

Coordinates in space are relative and require a different system of reference than Earth's Geodesic coordinates. As mentioned in the introduction, this paper will focus on viewpoint-independent spatial coordinate systems, rather than angular coordinates. Reference systems are largely defined by their central “origin” point, relative to which all coordinates are measured. Since space occupies 3-dimensional space, the orientation or rotation of the x-y-z axis planes is another important factor. The selection of a coordinate system therefore largely comes down to selecting an origin point and planar rotation.

Origin point

Spatial coordinate systems map locations in 3D space relative to a reference origin. Earth coordinate systems are considered absolute, fixed and unchanging (e.g., WSG84), anchored to the center of Earth, however no such universal origin exists for space. Instead, space coordinate systems use different origins depending on the context, including Earth (geocentric), the sun (heliocentric/baryocentric), or the center of the Milky Way (galactocentric, see **Table 1** for a full list). For example, a geocentric system is useful for plotting the trajectories of satellites in low earth orbit, whereas a heliocentric map is used in maps of the solar system (see **Table 2** for more examples). Given the fact there is no single agreed origin, it is important to adopt a flexible or compatible philosophy to origination.

Planar orientation

The orientation of axes is important to determine the relative location of bodies to the origin point. There is no absolute frame of reference for which way the universe is oriented, unlike on Earth where we use absolute North. The accepted approach is to anchor the x-y-z planes to very distant bodies that appear static. For example, the International Celestial Reference Frame ([ICRF3](#)) anchors the axis planes to distant super-bright and stable quasars.

Units

Selecting the appropriate units of measurement and degrees of precision is critical to ensuring a coordinate system remains functional across a wide range of use cases. A fundamental balance must always be struck between precision and numerical data size. Because data types like double-precision integers impose a strict resolution limit (roughly 1:9 billion), the higher the required precision, the smaller the maximum scale of the area that can be pragmatically mapped.

For example, if we mapped space using the exact same meter-level precision that we use for terrestrial [GPS](#), we would be limited to a maximum scale of approximately 0.95 light-years (ly), or about 0.3 parsecs. While this scale falls short of the nearest star, it is entirely sufficient for tracking the foreseeable future of human presence in space. Within this 0.95 ly radius, a meter-precision system can easily map the locations of satellites, spacecraft, and Voyager 1-the furthest human-made object from Earth. Furthermore, it easily accommodates the sub-meter

precision strictly required for aerospace engineering operations, such as calculating the exact coordinates of a planetary landing.

However, this meter-level precision cannot support the mapping of deep-space bodies. It would not even reach Alpha Centauri (~4 ly away), let alone span our wider galaxy or the universe as a whole. The natural solution for mapping at these macro-scales is to proportionally decrease precision to increase the navigable scale. Stepping down to kilometer-precision, for instance, extends the mapping reach to roughly 952 ly, which covers a significant local portion of our galaxy. Decreasing the resolution further to a 4-ly precision would permit the mapping of the entire known universe (~93 billion ly).

Ultimately, while the data capacity of coordinate units imposes hard mathematical limits, these boundaries rarely hinder practical use cases. At macro-scales, the digital precision of these coordinate systems far exceeds the actual precision of our instrumental measurements; for example, distance calculations for remote galaxies currently operate with an uncertainty margin between 1% and 10% (see Table 4).

Therefore, establishing a default resolution of 1 meter is the most pragmatic choice for mapping operational human spaceflight. When communicating these vast distances to the public or designing data visualizations, these highly precise units can simply be abstracted into Light Years (LY) to ensure the information remains accessible and easy to comprehend.

Instruments

The location of bodies in space are measured or calculated using a few different methods. Unlike Earth geography, mapping the universe is not a solved problem, with some significant challenges remaining. However, we have precise methods to create accurate and useful maps of the solar system for research, commercialisation, communication, and education. The main methods of collecting data to map space are detailed below, with their precision reducing the further away we get from Earth:

- The solar system is mapped by Direct Ranging using probe, satellite or earth-based telemetry (radar, laser, etc.)
- Nearby bodies (<1,000 LY, <1% precision) are mapped using the Parallax Method which triangulates the position of a body by comparing measurements taken at two different locations (e.g., throughout Earth's orbit round the sun)
- Nearby galaxies (<100M LY, ~5-10% precision) are mapped using the Standard Candles method which exploits knowledge of how bright a star is to calculate its distance based on how bright the star appears once the light reaches Earth
- The distant universe (>100M LY, ~10-20% precision) is mapped using Redshift which exploits our assumption that the galaxy is expanding uniformly to estimate the distance of stars again based on the wavelength of light that reaches the earth

Instrumental precision

The precision of the adopted space coordinate system will be lower than that required for mapping the surfaces of planetary bodies, such as Earth. For space mapping, there are more

practical limitations compared with Earth mapping, including lower instrumental sensitivity, and more approximate positioning due to less frequent surveillance.

Storing coordinate values

Coordinate systems need to store positions using large, high-precision numerical data types. Numbers are most commonly stored in computers as 32-bit floating-point precision numbers, however these numbers can only store a precision of up to 7 decimal places, with a maximum value around 3.4×10^{38} . Precise Earth coordinates often exceed these values, and such are often stored as 64-bit integers (e.g., [Google Earth](#)). Coordinates in space will therefore also need to represent coordinate values with at least this degree of precision. Indeed, developers of [3D space videogames](#) have already encountered this limitation. One consideration for data visualisation, however, is that most systems will need to render on a smaller scale, such as 32-bit numbers using methods like [floating-origins](#). Therefore, any such specification MUST support 64-bit precision coordinate values.

2D “X-Y” Coordinates

Space can be mapped in 2D, and this MUST be supported at a minimum. Solar systems and galaxies form from accretion discs and so are naturally distributed in two-dimensional, coplanar, space. Perhaps surprisingly, then, most of space can be mapped on a local scale in two dimensions. GeoJSON-like geometries, using the new coordinate reference system, may therefore map features like the centre of the Milky Way as a point, or the orbit of the Earth round the Sun as a circular polygon. Therefore, two “x” and “y” coordinates MUST be adopted as REQUIRED components of any feature, denoting position in x- and y-planes.

3D “Z” Coordinates

Space must, however, be mapped in 3D when looking at a multi-galactic or universal scale. While the distribution of features within a galaxy are coplanar, galaxies are randomly distributed across the universe in three-dimensional space, and each one is oriented/tilted at a different angle. While GeoJSON technically supports a third “altitude” coordinate, this exclusively refers to the height above/below Earth’s surface and so is purely used for projections including birds-eye views of earth, low earth orbit views, ([d3-geo](#)) and sky views ([d3-celestial](#)). GeoJSON-like 3D geometries can be made by adopting “z” as the third coordinate on the new coordinate reference system. Therefore, a third “z” coordinate MUST be adopted as an OPTIONAL feature component to denote space in the z-plane.

4D “M” Coordinates

Space has a lot more motion than Earth geographies do, and this must be supported in maps. GeoJSON makes a tacit mention to the informal practice of extending the coordinate system with *M*, a fourth component representing datetime, though makes it clear that it is NOT officially supported. There are two reasons that a fourth “*M*” time component SHOULD be adopted as an OPTIONAL component: easing data integration and supporting space time.

Adopting a fourth time dimension would make data integration more practical. All bodies in space are perpetually in motion, measures of their location will change over time and so are inherently time-based. For example, the earth rotates $\sim 360^\circ$ per day, orbits the Sun every ~ 365 days, and to a lesser extent is influenced by Galactic factors such as the rotation of the Milky Way, and the expansion of the universe. Therefore, including time information at the feature level would aid in the quantitative easing of space positions across time. For example, a map could have a specific time, such as Epoch, so that the locations of all bodies in the map could be synchronised to this time (e.g., resolving the location of two orbital bodies that have positional data recorded from two different times). A further example is on “lookback” time, which describes the phenomenon whereby all of our information about locations of bodies arrives at Earth at a finite speed (the speed of light), and therefore the positions of bodies is inherently historical. This effect becomes more dramatic as we observe further away galaxies, and may be relevant for producing maps from data that is highly time-heterogeneous. Supporting a time component will therefore help aid data processing, and account for velocities and lookback time.

Furthermore, adopting an M dimension would also support mapping features with variable spacetime. This could either allow for simultaneous mapping of time-heterogeneous features. Einstein’s Special and General Theories of Relativity abandon the notion that time is a universal and fixed property of all bodies in the universe, but instead time may be experienced differently based on a body’s movement in space. Specifically, a body that moves faster in space also moves forwards in time, resulting in the other bodies around them appearing to “age” faster than themselves. This phenomenon is further [explained elsewhere](#), but the implication for mapping space is clear; space maps on a sufficiently large scale MUST allow for bodies to have relative time. For example, fast moving bodies experience the passage of time more slowly, and as a result GPS satellites in Earth’s orbit run “slow” and personnel aboard the International Space Station age slower (~ 14 seconds per year). Incorporating time information at the feature level would allow for relativistic spacetime effects or simulation.

Summary

In summary, there is no existing space mapping data interchange format, and the rest of the present paper will propose one. The present paper proposes adapting GeoJSON, by building on its existing structures and definitions, expanding them for the purposes of mapping space.

We arrive at a set of requirements for a data interchange format for space maps:

Core Architectural Requirements

- **Viewpoint Independence:** The system must use a spatial reference system that does not rely on a specific viewer’s perspective.
- **Backward Compatibility:** Must remain compatible with standard GeoJSON for terrestrial surface mapping, as well as orbital reference systems for satellites.

- **Flexible Origins:** Must support extra-terrestrial points of origin (e.g., heliocentric, galactocentric) rather than being strictly Earth-centered.
- **Standardized Axes:** Must adopt a standardized coordinate reference system utilizing three oriented axis planes.
- **Spacetime Support:** Must support relative spacetime mechanics to allow for the realistic modeling of time dilation and fast-moving bodies.

Strict Technical Directives (The "MUSTs")

- **Standards Compliance:** The implementation **MUST** follow GeoJSON (RFC7946) using a "least-required modification" policy, and terminology **MUST** be interpreted exactly as described in RFC 2119.
- **Immutability:** To function as a universal standard, implementations of the AstroJSON format **MUST NOT** be modified from the specification.
- **Reference Frame:** Astronomical geometric vectors **MUST** be expressed against **ICRF3** (the International Celestial Reference Frame).
- **Geometry Format:** Because ADQL (Astronomical Data Query Language) lacks flexibility, spatial vectors **MUST** be represented as GeoJSON-like geometries.
- **Data Precision:** The system **MUST** support **64-bit precision** numerical data types to handle vast cosmic distances without losing accuracy.
- **Dimensionality Rules:**
 - * **2D (Required):** The format **MUST** include 'x' and 'y' coordinates at a minimum.
 - **3D (Optional):** The format **SHOULD** include an optional 'z' coordinate to map multi-galactic or universal scales.
 - **4D (Optional):** The format **SHOULD** include an optional 'M' (time) coordinate to account for orbital motion, lookback time (speed of light), and Einsteinian relativity.

4. Implementation

This section will propose an implementation of an interchange format for view-point independent space map data. The GeoJSON specification (RFC7946) will be followed where possible, following a policy of least-required modification. An implementation of a space data interchange format **MUST NOT** be modified since to function as a standard all required directions **MUST** be followed.

Astronomical geometric vectors are expressed as coordinate vectors against ICRF3. Astronomical Data Query Language (ADQL) used by Table Access Protocol (TAP) services including the Gaia Archive (see **Table 4** for list of major astronomical databases) allows users to specify spatial queries to return discovered stars at a given geometric point, circle, or polygon area in space. For example, `CIRCLE('ICRS', center_ra, center_dec, radius_degrees)`. However, the ADQL specification is not as flexible as a GeoJSON-like

geometry specification, and so is only practically designed for representing simple geometric space. Therefore, astronomical vectors SHOULD be represented as GeoJSON-like geometries.

Figure 2. Minimal example of an “AstroJSON” GeoJSON-like geometry schema with ADQL-like coordinate vectorisation.

```
json
{
  "type": "Feature",
  "properties": {
    "name": "Andromeda Galaxy"
  },
  "geometry": {
    "type": "Circle",
    // Galaxy shape in galaxy-scale appropriate coordinate system
    "center_ra": 10.684,
    "center_de": 41.269,
    "radius_degrees": 1.58
  },
}
```

5. Appendix

Table 1. Existing standards used to map the locations of bodies in space. These demonstrate that there is no standard one-size-fits-all approach to mapping space.

Name	Type	Description	Examples
WGS84 (World Geodetic System 1984)	Earth-Fixed Geodetic	Latitude, longitude, and altitude relative to a reference ellipsoid. Rotates with Earth.	Google Earth, GeoJSON, GPS receivers
ECEF (Earth-Centered Earth-Fixed)	Earth-Fixed Cartesian	X, Y, Z coordinates in kilometers, anchored to Earth's center. Rotates with Earth. The Cartesian equivalent of WGS84.	GPS internals, geodesy, transforms
EME2000 / J2000 (Earth Mean Equator and Equinox of J2000)	Earth-Centered Inertial	X, Y, Z coordinates fixed to Earth's orientation at noon on Jan 1, 2000. Does not rotate with Earth.	NASA mission planning, precision astrodynamics
GCRF / ICRF (Geocentric/International Celestial Reference Frame)	Inertial Reference Frame	The most precise inertial frame, fixed relative to distant quasars. Nearly identical to J2000 but formally defined by the IAU.	High-precision astronomy, VLBI, deep space navigation
TEME (True Equator Mean Equinox)	Earth-Centered Inertial (of-date)	Hybrid inertial frame using the current equator but an averaged equinox	SGP4 propagator output

		direction. Exists because SGP4 was built around it.	
TOD (True of Date)	Earth-Centered Inertial (of-date)	Inertial frame using both the true equator and true equinox at the current moment. More rigorous than TEME.	Precision orbit determination
MOD (Mean of Date)	Earth-Centered Inertial (of-date)	Inertial frame using averaged (mean) equator and equinox at the current moment. Smooths out short-term wobbles.	Intermediate frame for transforms
TLE (Two-Line Element Set)	Orbital Element Encoding	A compact text format encoding six Keplerian orbital elements plus drag terms. Designed for punch cards in the 1970s. Must be used with SGP4 propagator.	CelesTrak, Space-Track.org, satellite trackers
State Vector	Orbital Representation	Position (X, Y, Z) and velocity (Vx, Vy, Vz) at a specific epoch. Six numbers plus timestamp. More precise than TLEs.	LeoLabs, NASA ephemerides, conjunction analysis
Keplerian Elements	Orbital Representation	Six parameters describing an orbit's shape and orientation: semi-major axis, eccentricity, inclination, RAAN, argument of perigee, true anomaly.	Mission design, orbit visualization, TLE internals
Equatorial Coordinates (Right Ascension / Declination)	Celestial Coordinates	Angular coordinates on the celestial sphere. RA measured in hours/minutes/seconds eastward from vernal equinox; Dec in degrees from celestial equator.	Star catalogs, telescope pointing, astronomy
Galactic Coordinates	Celestial Coordinates	Angular coordinates relative to the Milky Way's center and plane. Longitude (l) and latitude (b).	Galactic astronomy, radio surveys

Table 2. Examples of existing space maps and their technology. This demonstrates that there is no single adopted standard for mapping space.

Name	Rendering method	Origin	Mapping system
Google Earth Sky View	Raster Projection + Vector	Geocentric	EME2000
NASA Eyes on the Solar System	Vector	Heliocentric	
Johns Hopkins Map of the Universe	Raster	Geocentric	Equatorial (RA/Dec)
LeoLabs Low Earth Orbit Map	Vector (+ Raster Projection for	Geocentric	EME2000

	reference)		
NASA Sky Map	Raster	Geocentric	Equatorial (RA/Dec)
SatelliteMap.space	Vector	Geocentric	ECEF
Elite Dangerous star map *	Vector	Heliocentric	Hierarchical

Note: Elite Dangerous is an example of a videogame that maps a fantasy universe loosely based on our universe, with realistic physics, and illustrates the kinds of challenges we face when developing universe-scale maps. Other similar examples include the development diaries for EvE Online, Star Citizen, and even Microsoft Flight Simulator.

Table 3. Summary of GeoJSON geometry types. See RFC7946 section 3.1 for more information.

Type	Type
Point	Single position
MultiPoint	Array of positions
LineString	Array of 2+ positions (a path)
MultiLineString	Array of LineStrings
Polygon	Array of linear rings (first = exterior, rest = holes)
MultiPolygon	Array of Polygons
GeometryCollection	Heterogeneous mix of geometries

Table 4. Major astronomical databases

Archive	Focus	Scale	Key Access
MAST	Optical/UV/NIR (Hubble, JWST, TESS)	200+ TB	TAP, REST API
HEASARC	X-ray/Gamma (Chandra, Fermi)	160+ TB	TAP, XAmin portal
IRSA	Infrared (Spitzer, WISE, 2MASS)	8+ PB	Gator, TAP
Gaia Archive	Astrometry	~10 TB	TAP+ with ADQL
PDS	Planetary missions	Federated	PDS4 XML, REST

VizieR	Published catalogs	25,839 catalogs	TAP, Cone Search
---------------	--------------------	--------------------	------------------

6. Bibliography

Thorne, C. (2005). Using a Floating Origin to Improve Fidelity and Performance of Large, Distributed Virtual Worlds. *4th International Conference on Cyberworlds*.